

VBuddy — Software Engineering Showcase

VBuddy ist eine KI-gesteuerte Chat-Applikation, in der ein virtueller Freund (Virtual Buddy) ein eigenständiges Leben führt: Er plant seinen Tag, hat Bedürfnisse, schreibt Blogartikel und reagiert kontextbezogen auf Gespräche. Das Projekt demonstriert den durchgängigen Aufbau einer modernen, KI-integrierten Anwendung — von der Architektur über das Deployment bis zur Testbarkeit.

Inhaltsverzeichnis

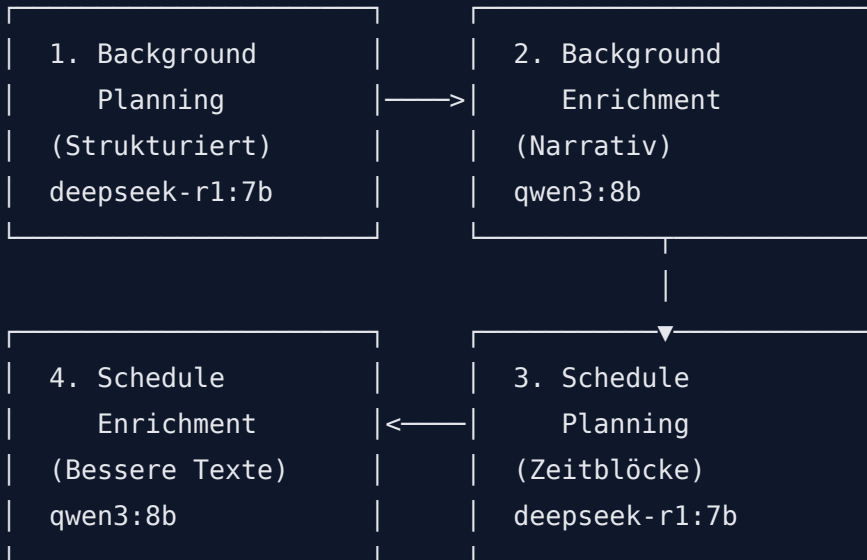
1. [Multi-Agent-Orchestrierung](#)
 2. [Autonome Tagesplanung](#)
 3. [Interaktive Tagesplanung via Chat](#)
 4. [RAG: Retrieval Augmented Generation](#)
 5. [Modellstrategie: Aufgabenspezifische LLM-Selektion](#)
 6. [Tool-Augmented Generation: Websuche](#)
 7. [Systemintegration: WordPress-Publishing-Pipeline](#)
 8. [Teststrategie: Qualitätssicherung für KI-Komponenten](#)
 9. [Technologie-Stack](#)
 10. [Cloud-Native Deployment: Containerisierung](#)
 11. [On-Premise LLM-Betrieb mit Ollama](#)
 12. [Architektur: Wartbarkeit & Testbarkeit](#)
 13. [Frontend-Architektur](#)
 14. [Datenbank-Design & Migrationen](#)
 15. [User Management & Security](#)
-

1. Multi-Agent-Orchestrierung

VBuddy basiert auf **mehrstufigen, orchestrierten Agenten-Pipelines**, in denen spezialisierte LLM-Aufrufe sequenziell aufeinander aufbauen und Zwischenergebnisse als Kontext in nachfolgende Verarbeitungsstufen einfließen.

Background-Pipeline (4 Stufen)

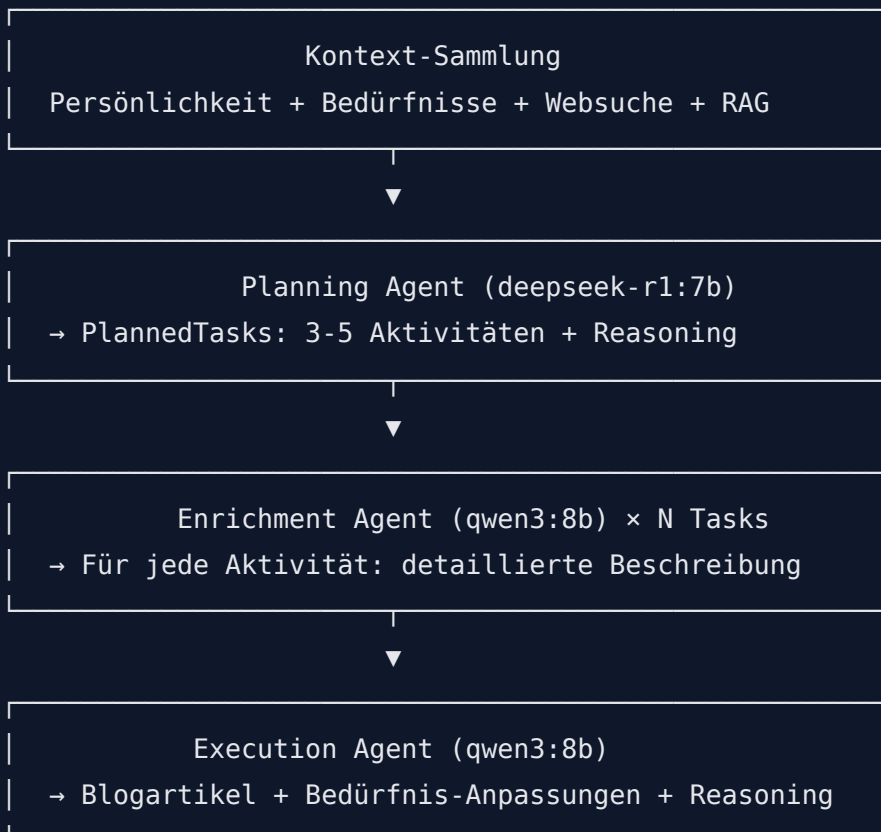
Die Erstellung einer Hintergrundgeschichte durchläuft vier sequenzielle LLM-Aufrufe, wobei jeder Schritt auf dem Ergebnis des vorherigen aufbaut:



Stufe 1 generiert ein strukturiertes Charakterprofil (Alter, Herkunft, Hobbys, Ängste, Biografie). **Stufe 2** verwandelt die strukturierten Daten in einen lebendigen Erzähltext. **Stufe 3** erstellt basierend auf dem Narrativ einen lückenlosen 24h-Wochenplan. **Stufe 4** verbessert die Formulierungen im Stundenplan passend zur Persönlichkeit.

Task-Pipeline (Planning → Enrichment → Execution)

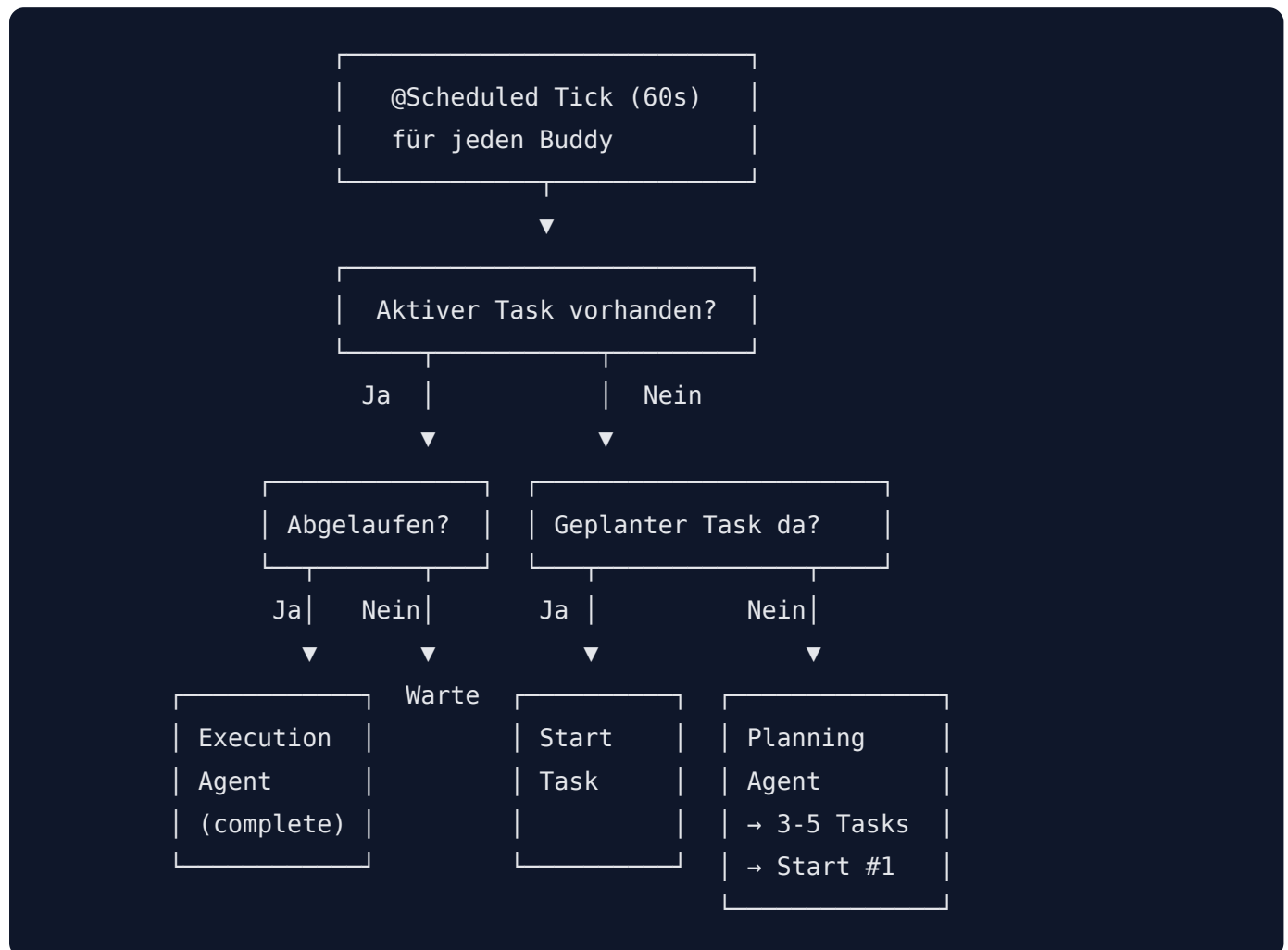
Die tägliche Aktivitätsplanung orchestriert drei Agenten:



2. Autonome Tagesplanung

Der VBuddy führt ein eigenständiges Leben — ohne dass der Nutzer eingreifen muss. Ein `@Scheduled` - Lifecycle-Tick (alle 60 Sekunden) orchestriert das gesamte Tagesgeschehen automatisch.

Lifecycle-Loop



Kontext-Sammlung vor der Planung

Bevor der Planning Agent aktiv wird, sammelt der `PlanningAgentService` umfangreichen Kontext aus verschiedenen Quellen:

Kontextquelle	Inhalt	Zweck
Persönlichkeit	Name, Charakterbeschreibung	Konsistente Aktivitäten
Bedürfnisse	5 Needs mit aktuellem Wert (0-100)	Priorisierung (hoher Hunger → Essen)
Letzte 10 Tasks	Titel, Ort, Zeit, Dauer	Wiederholungen vermeiden
Websuche	Lokale Events + Geschäfte/Restaurants	Reale Orte und Veranstaltungen
RAG-Kontext	Semantisch relevante vergangene Erlebnisse (Top 10)	Langzeitgedächtnis
Wochenplan	Regulärer Tagesablauf (Arbeit, Freizeit, Schlaf)	Rahmen für Aktivitäten
Tagestyp	Wochentag vs. Wochenende	Strukturierter vs. freier Tag

Execution: Von der Aktivität zum Blogartikel

Wenn ein Task zeitlich abgelaufen ist, übernimmt der Execution Agent:

1. **LLM-Aufruf** — Generiert Blogartikel (Ich-Perspektive) + Bedürfnis-Anpassungen
2. **Bedürfnisse anpassen** — z.B. Joggen: EXERCISE -25, HUNGER +5
3. **Blog lokal speichern** — Titel + Inhalt in der DB
4. **Bildersuche** — SearxNG findet 3 passende Bilder zum Thema
5. **WordPress-Publish** — Artikel mit Bildern auf dem echten Blog veröffentlichen
6. **Standort aktualisieren** — Buddy-Location auf den Task-Ort setzen
7. **Embedding** — Abgeschlossenen Task für RAG vektorisieren
8. **AI Decision Log** — Entscheidung + Reasoning protokollieren

3. Interaktive Tagesplanung via Chat

Neben der autonomen Planung kann der Nutzer den Tagesplan des VBuddy **über natürliche Sprache im Chat** beeinflussen. Dabei bleiben Autonomie und Persönlichkeit des VBuddy erhalten — er kann Vorschläge annehmen, ablehnen oder Rückfragen stellen.

Dynamischer System-Prompt

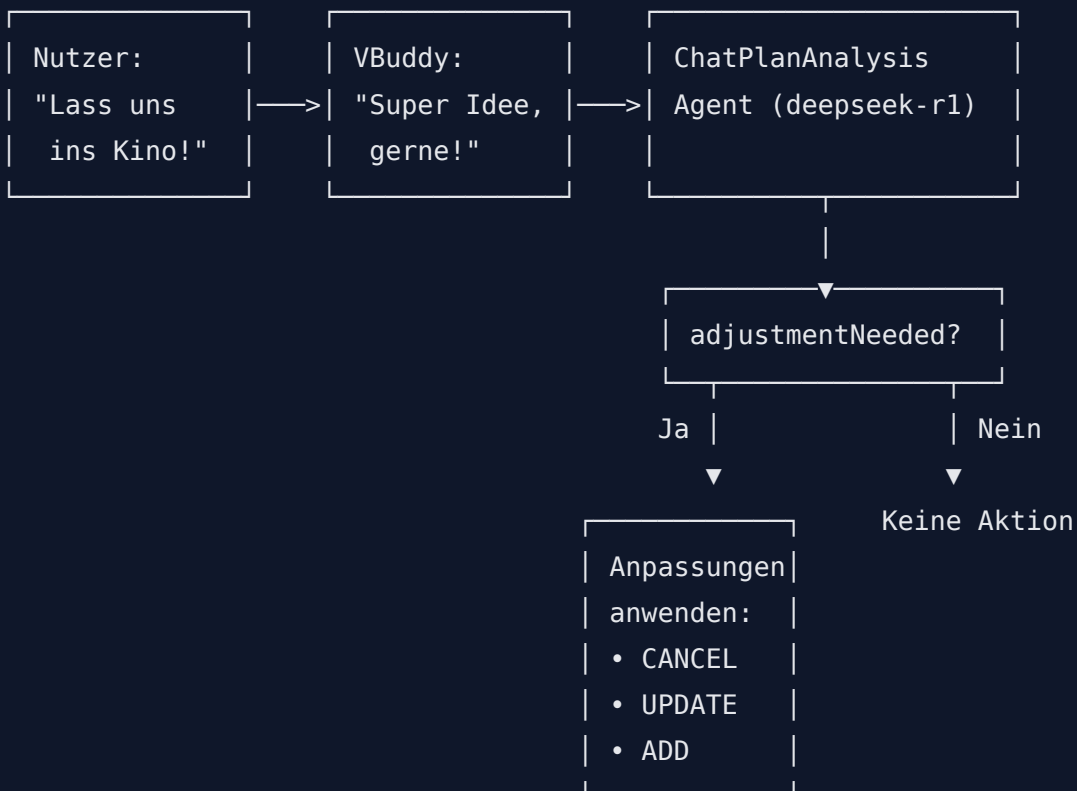
Der Chat-System-Prompt wird bei jeder Nachricht **dynamisch aus dem aktuellen Zustand** zusammengebaut:

System-Prompt (dynamisch)	
Rollenanweisung	"Du bist Max, ein virtueller Buddy..."
Aktuelle Uhrzeit	2026-03-21 14:30
Persönlichkeit	"neugierig, sportlich, introvertiert"
RAG-Kontext	Semantisch relevante Erinnerungen
Aufenthaltort	"Stadtpark Nürnberg"
Bedürfnisse	HUNGER: 65, EXERCISE: 70, ...
Aktuelle Aktivität	"Joggen im Stadtpark (seit 14:00)"
Letzte 5 Tasks	Erledigte Aktivitäten des Tages
Geplante Tasks	Kommende Aktivitäten mit Zeiten
Regeln	Sprache, Planänderungs-Handling

Der VBuddy kennt dadurch seinen vollständigen aktuellen Zustand und kann kontextbezogen auf Vorschläge reagieren.

Chat-Plan-Analyse-Pipeline

Nach jeder Chat-Nachricht wird automatisch ein **Analyse-Agent** ausgelöst, der prüft, ob eine Planänderung vereinbart wurde:



Drei Arten von Planänderungen

Aktion	Beschreibung	Beispiel
CANCEL	Bestehenden Task absagen	"Es regnet, lass das Joggen ausfallen" → VBuddy stimmt zu → Task wird auf ABORTED gesetzt
UPDATE	Bestehenden Task ändern	"Geh doch lieber ins Café statt ins Restaurant" → Titel, Ort, Beschreibung werden aktualisiert
ADD	Neuen Task hinzufügen	"Hast du Lust auf Kino um 15 Uhr?" → VBuddy sagt ja → Neuer PLANNED-Task wird erstellt

Schutz vor False Positives

Der Analyse-Agent unterscheidet bewusst zwischen:

- **Smalltalk** → Keine Planänderung ("Wie geht's?", "Schönes Wetter")
- **Rückfragen** → Keine Planänderung ("Welchen Film meinst du?")
- **Expliziter Vorschlag + Zustimmung** → Planänderung wird angewendet
- **Vorschlag + Ablehnung** → Keine Planänderung ("Nee, ich bleibe lieber hier")

Alle angewendeten Planänderungen werden im **AI Decision Log** protokolliert — mit Reasoning, warum die Änderung erkannt wurde.

4. RAG: Retrieval Augmented Generation

VBuddy nutzt **Retrieval Augmented Generation** als semantisches Langzeitgedächtnis. Über pgvector-Embeddings (768 Dimensionen) werden vergangene Erlebnisse und die Hintergrundgeschichte vektorisiert und bei Bedarf kontextbezogen abgerufen.

Embedding-Quellen

Zwei Kategorien von Inhalten werden vektorisiert und in pgvector gespeichert:

Quelle	Wann embedded?	Metadata	Inhalt des Vektors
Hintergrundgeschichte	Nach Background-Generierung (Stufe 2)	buddy_id , type=background	Narrativer Erzähltext (Biografie, Persönlichkeit)
Tasks	Bei Status-Wechsel (PLANNED → IN_PROGRESS → COMPLETED / ABORTED)	buddy_id , type=task , task_id , status	Titel, Status, Zeitpunkt, Dauer, Ort, Beschreibung

Embedding-Zeitpunkte im Task-Lifecycle

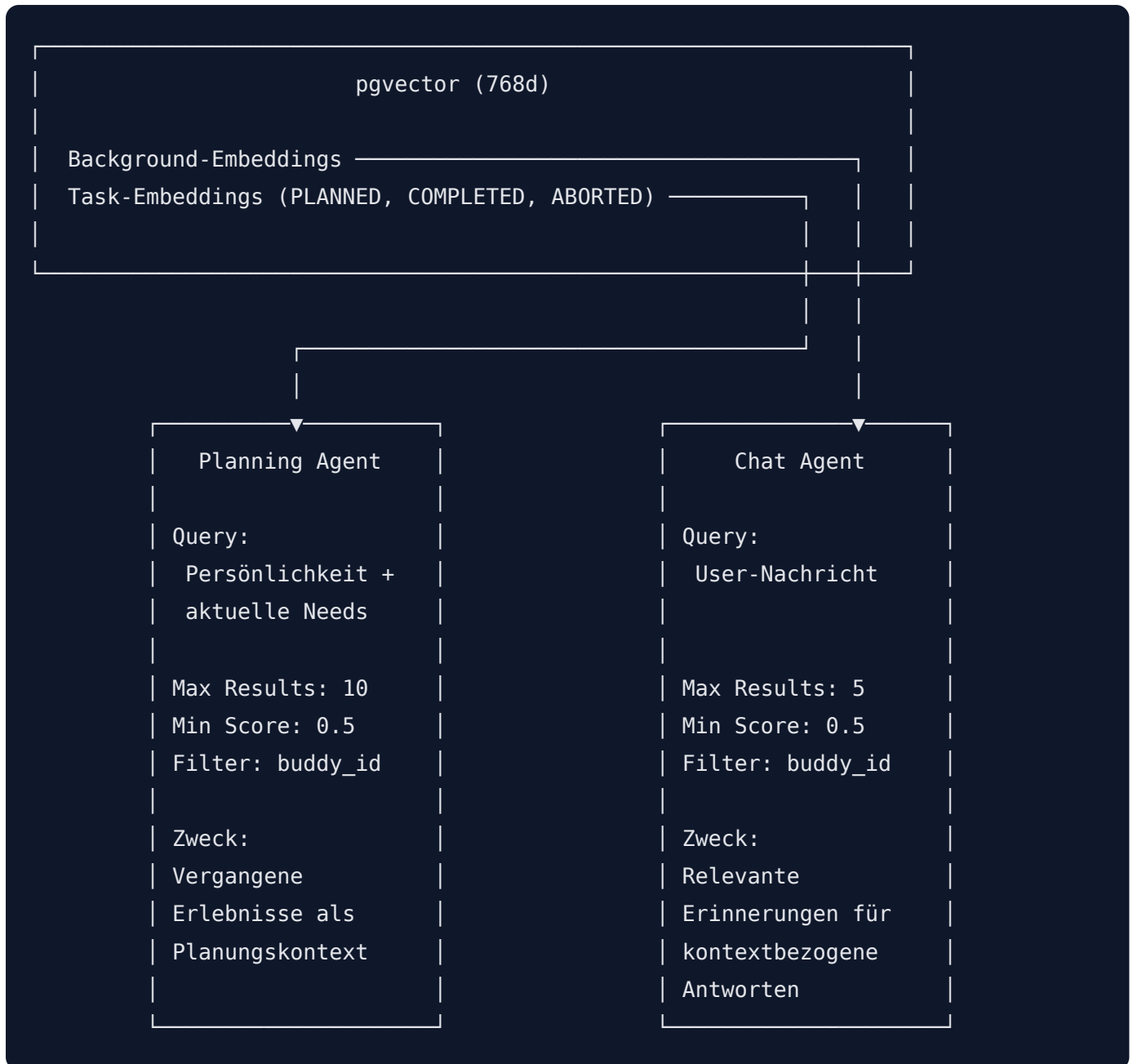
Tasks werden nicht nur einmal, sondern **bei jedem Status-Wechsel** neu embedded — der Vektor wird aktualisiert (altes Embedding wird per `task_id`-Filter gelöscht):

```
PLANNED — embedTask() —> pgvector (Geplant: Joggen im Park, 17:00)
|
▼
IN_PROGRESS — embedTask() —> pgvector (Aktiv: Joggen im Park, seit 17:00)
|
▼
COMPLETED — embedTask() —> pgvector (Erledigt: Joggen im Park, 45 min)
```

Auch über den Chat geänderte oder abgesagte Tasks (`ABORTED`) werden embedded — so fließen auch Planänderungen ins Langzeitgedächtnis ein.

Abruf: Wo greift RAG?

RAG-Kontext wird an **zwei zentralen Stellen** in den LLM-Prompt injiziert:



Konkretes Beispiel

Situation: Der Nutzer fragt im Chat: "Warst du schon mal im Café am Hauptmarkt?"

1. **Embedding:** Die User-Nachricht wird vektorisiert
2. **Suche:** pgvector findet semantisch ähnliche Einträge für diesen Buddy:
 - Task "Kaffee trinken im Café am Hauptmarkt" (COMPLETED, vor 3 Tagen)
 - Task "Einkaufen in der Altstadt" (COMPLETED, vor 1 Woche)
3. **Injection:** Die gefundenen Erlebnisse werden als "Relevanter Hintergrund und Erfahrungen" in den System-Prompt eingefügt
4. **Antwort:** Der VBuddy kann konkret antworten: "Ja, ich war vor drei Tagen dort! Der Cappuccino war super..."

Technische Umsetzung

```
// EmbeddingService – Abruf mit Buddy-Filter und Schwellenwert
EmbeddingSearchRequest request = EmbeddingSearchRequest.builder()
    .queryEmbedding(queryEmbedding)
    .maxResults(maxResults)
    .minScore(0.5) // Nur relevante Treffer
    .filter(MetadataFilterBuilder
        .metadataKey("buddy_id")
        .isEqualTo(buddyId.toString())) // Nur eigene Erinnerungen
    .build();
```

- **Buddy-Isolation:** Jeder VBuddy hat sein eigenes Gedächtnis — kein Buddy sieht die Erinnerungen eines anderen
- **Schwellenwert 0.5:** Nur semantisch wirklich relevante Treffer werden zurückgegeben
- **Idempotentes Re-Embedding:** Bei Task-Updates wird das alte Embedding gelöscht und ein neues erstellt

5. Modellstrategie: Aufgabenspezifische LLM-Selektion

VBuddy setzt gezielt **unterschiedliche LLM-Modelle** für spezifische Aufgabenprofile ein — die Modellwahl erfolgt anhand der jeweiligen Stärken in Bezug auf Reasoning, Textgenerierung und Dialogfähigkeit:

Aufgabe	Modell	Stärke	Temperature
Planung & Logik	deepseek-r1:7b	Reasoning, strukturierte Entscheidungen	0.7
Kreatives Schreiben	qwen3:8b	Blogpost-Generierung, Erzähltexte	0.7
Konversation	llama3.1:8b	Natürlicher Dialog, Empathie	0.7
Stundenplanung	qwen3:8b	Strukturierte Zeitblöcke	0.8
Embedding	nomic-embed-text	Semantische Vektorisierung (768d)	—

Begründung der Modellwahl

- **deepseek-r1** für Planung: Das Modell verfügt über einen internen Chain-of-Thought-Mechanismus, der für logische Ablaufplanung und konsistente Entscheidungsfindung optimiert ist.
- **qwen3** für Ausführung/Enrichment: Kreative Textgenerierung mit guter Instruktionsbefolgung — ideal für Blogartikel und lebendige Beschreibungen.
- **llama3.1** für Chat: Starke Performance bei natürlichem Dialog mit kurzer Latenz.
- **nomic-embed-text** für RAG: Effizientes Embedding-Modell mit 768 Dimensionen, das lokal auf Ollama läuft.

Strukturierte Ausgabe via LangChain4j

Alle AI Services nutzen LangChain4j's **Structured Output** — die LLM-Antworten werden automatisch in typisierte Java-Records geparkt:

```
public record PlannedTasks(  
    @Description("Die geplanten Aktivitäten in chronologischer Reihenfolge")  
    List<PlannedTask> tasks,  
    @Description("Kurze Begründung, warum diese Aktivitäten gewählt wurden")  
    String reasoning  
) {}
```

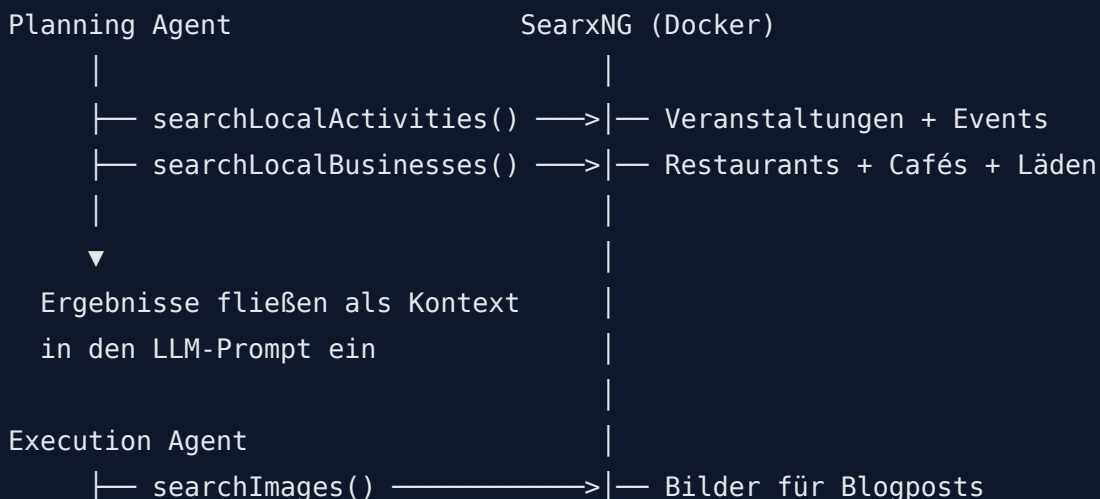
Jeder AI Service ist ein **reines Interface** mit `@SystemMessage` und `@UserMessage` — LangChain4j generiert die Implementierung zur Laufzeit:

```
public interface PlanningAiService {  
    @SystemMessage("Du bist der interne Planungs-Agent eines virtuellen  
    Buddys...")  
    @UserMessage("Plane die nächsten Aktivitäten...")  
    PlannedTasks planTasks(@V("personality") String personality, ...);  
}
```

6. Tool-Augmented Generation: Websuche

VBuddy integriert eine **SearxNG-Metasuchmaschine** als Tool-Augmentation, um den Agenten Zugriff auf aktuelle, lokale Informationen bereitzustellen.

Architektur



Kontextanreicherung

Die Websuche-Ergebnisse werden als strukturierter Text in den Planungs-Prompt integriert:

```
## Veranstaltungen & Events
1. Nürnberger Bardentreffen 2026
   Open-Air-Musikfestival in der Altstadt...
   Quelle: https://...

## Geschäfte, Restaurants & Cafés
1. Café am Hauptmarkt
   Gemütliches Café mit hausgemachten Kuchen...
   Quelle: https://...
```

Der Planning Agent kann dadurch **echte Ortsnamen, Veranstaltungen und Geschäfte** in seine Aktivitätsplanung einbauen — der VBuddy erlebt eine Welt, die auf realen Daten basiert.

7. Systemintegration: WordPress-Publishing-Pipeline

Der VBuddy veröffentlicht seine Blogartikel automatisch auf einem echten WordPress-Blog unter <https://elitegames.v6.rocks/vbuddy-blog/>.

Ablauf der Blog-Veröffentlichung

```
Execution Agent
|
├─ 1. LLM generiert Blogartikel (Titel + Inhalt, Ich-Perspektive)
|
├─ 2. SearxNG-Bildersuche (3 Bilder passend zum Thema)
|
├─ 3. WordPress REST API:
|   └─ Bilder hochladen → /wp/v2/media
|   └─ Bilder als WordPress-Blöcke einbetten
|   └─ Post publizieren → /wp/v2/posts (status: publish)
|
└─ 4. Blog-Eintrag in lokaler DB speichern
```

Technische Details

- **Authentifizierung:** HTTP Basic Auth über die WordPress REST API v2
- **Bildintegration:** Bilder werden als WordPress-Gutenberg-Blöcke (`wp:image`) eingebettet

- **Graceful Degradation:** Wenn WordPress nicht erreichbar ist oder Credentials fehlen, läuft die Anwendung ohne Blog-Veröffentlichung weiter
- **Konfigurierbar:** WordPress-URL, Username und Password über Umgebungsvariablen

8. Teststrategie: Qualitätssicherung für KI-Komponenten

VBuddy implementiert eine **dreistufige Teststrategie**, die klassische Unit-Tests, LLM-Output-Validierung und Service-Integrationstests umfasst.

Teststufen

Stufe 1: Unit Tests

`mvn test`

- Service-Logik mit gemockten AI-Interfaces
- Controller-Tests
- Keine externen Abhängigkeiten
- Laufen bei jedem Build

Stufe 2: LLM Integration Tests

`mvn test -Dgroups=llm`

- Testen den tatsächlichen LLM-Output
- Strukturvalidierung (Felder, Formate, Wertebereiche)
- Plausibilitätsprüfung (Hunger → Essen, Ich-Perspektive)
- Konsistenzprüfung (chronologische Reihenfolge, 24h-Abdeckung)
- Benötigen laufendes Ollama

Stufe 3: Service Integration Tests

`mvn test -Dgroups=
integration`

- Testen externe Service-Anbindungen (SearxNG)
- Ergebnisqualität und -struktur
- Benötigen laufende externe Services

Testbare AI-Architektur

Die AI Services sind als **reine Interfaces** definiert, die per `AiServices.builder()` sowohl im Produktivcode (Spring Beans) als auch in Tests (standalone, ohne Spring Context) instanziiert werden können:

```
// In Tests – ohne Spring, ohne DB, ohne Docker
ChatLanguageModel model = OpenAiChatModel.builder()
    .baseUrl("http://localhost:11434/v1")
    .modelName("deepseek-r1:7b")
    .build();

PlanningAiService service = AiServices.builder(PlanningAiService.class)
    .chatLanguageModel(model)
    .build();

PlannedTasks result = service.planTasks(...);
assertThat(result.tasks()).hasSizeBetween(3, 5);
```

Beispiele für LLM-Output-Assertions

Test	Prüft
Tasks sind chronologisch geordnet	Zeitliche Konsistenz des Planungs-Agents
Hoher Hunger → Essens-Aktivität	Bedürfnis-Reaktivität
Blog in Ich-Perspektive	Einhaltung der System-Message
NeedAdjustments im Bereich -30 bis +10	Regelkonformität
Wochenplan deckt 00:00-00:00 ab	24h-Vollständigkeit
Zeitblöcke sind lückenlos	Strukturelle Konsistenz
Smalltalk ≠ Planänderung	False-Positive-Vermeidung

Build-Konfiguration

LLM- und Integration-Tests sind aus dem regulären Build ausgeschlossen:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <excludedGroups>llm,integration</excludedGroups>
  </configuration>
</plugin>
```

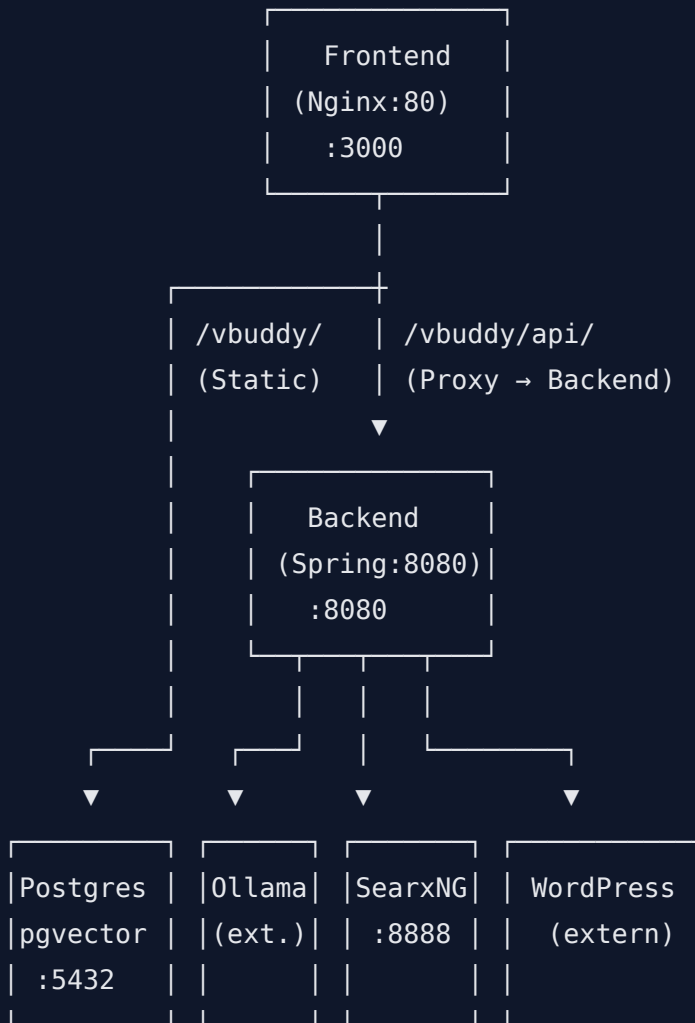
9. Technologie-Stack

Schicht	Technologie	Version
Frontend	React + TypeScript	19 + 5.7
Build (Frontend)	Vite	6.0
Routing	React Router	7.1
Backend	Spring Boot	3.4
Sprache	Java	21
Build (Backend)	Maven	3.9
AI/LLM-Framework	LangChain4j	0.36.2
LLM-Provider	Ollama (selfhosted)	latest
Datenbank	PostgreSQL + pgvector	17
Migrationen	Flyway	latest
Webserver	Nginx	alpine
Websuche	SearxNG	latest
Containerisierung	Docker Compose	latest

10. Cloud-Native Deployment: Containerisierung

Die gesamte Anwendung ist als **Docker-Compose-Stack** konzipiert — ein `docker compose up --build` startet alle Services.

Service-Architektur



Multi-Stage Dockerfiles

Beide Dockerfiles nutzen **Multi-Stage-Builds** für minimale Image-Größen:

Backend: Maven-Build auf `eclipse-temurin:21` → Runtime auf `eclipse-temurin:21-jre`

Frontend: `npm run build` auf `node:20-alpine` → Auslieferung via `nginx:alpine`

Netzwerk-Isolation

```

networks:
  vbuddy_internal_net:
    internal: true          # Kein Internetzugang – nur interne Kommunikation
  container_app_network:
    external: true         # Backend erreicht externe Dienste (Ollama,
WordPress)
  
```

- **Postgres und SearxNG** sind nur im internen Netzwerk erreichbar
- **Backend** hat Zugang zum internen Netzwerk (DB, SearxNG) und zum externen Netzwerk (Ollama, WordPress)

- **Healthchecks** stellen sicher, dass der Backend-Service erst startet, wenn PostgreSQL bereit ist
-

11. On-Premise LLM-Betrieb mit Ollama

VBuddy läuft vollständig mit **lokal betriebenen LLMs** — keine Cloud-APIs, keine tokenbasierte Abrechnung, volle Datenhoheit.

Ollama-Setup

Ollama stellt eine **OpenAI-kompatible API** bereit (`/v1/chat/completions`), die von LangChain4j über den `OpenAiChatModel` -Builder angesprochen wird. Dadurch ist der LLM-Provider austauschbar — ein Wechsel zu OpenAI, Azure oder einem anderen Provider erfordert nur eine Konfigurationsänderung:

```
vbuddy:
  ai:
    planning:
      base-url: ${PLANNING_LLM_BASE_URL:http://localhost:11434/v1}
      api-key: ${PLANNING_LLM_API_KEY:ollama}
      model-name: ${PLANNING_LLM_MODEL:deepseek-r1:7b}
```

Vorteile des On-Premise-Betriebs

- **Kostenkontrolle** — unbegrenzte Inferenz ohne tokenbasierte Abrechnung
 - **Datenhoheit** — sämtliche Daten verbleiben im lokalen Netzwerk
 - **Modell-Flexibilität** — Modellwechsel ausschließlich über Konfiguration, ohne Code-Änderungen
 - **Offline-Fähigkeit** — die Anwendung ist ohne Internetverbindung lauffähig (ausgenommen Websuche und WordPress-Publishing)
 - **Reproduzierbarkeit** — deterministische Modellversionen, keine unkontrollierten Provider-seitigen Updates
-

12. Architektur: Wartbarkeit & Testbarkeit

Die Backend-Architektur ist konsequent auf **lose Kopplung, klare Verantwortlichkeiten und Testbarkeit** ausgelegt. Jede Schicht hat eine definierte Rolle, und Abhängigkeiten zeigen immer nach innen.

Schichtarchitektur

Controller-Schicht			
7 REST Controller + DTOs (Request/Response Records)			
GlobalExceptionHandler + Bean Validation			
Service-Schicht			
16 Services – Business-Logik, Orchestrierung, Fehlerbehandlung			
AI Service	Embedding	SearxNG Search	WordPress
Interfaces	Service	Service	Service
(8 Stück)	(RAG)	(Websuche)	(Blog-Publish)
Repository-Schicht			
Spring Data JPA Interfaces (9 Repositories)			
PostgreSQL 17 + pgvector			

Entkopplung der AI-Schicht

Das zentrale Architekturprinzip: **AI Services sind reine Interfaces** — keine Implementierungsklassen, keine Vererbung. LangChain4j generiert die Implementierung zur Laufzeit, die `AiConfig` bindet sie als Spring Beans:



Warum das wichtig ist:

- **Testbarkeit:** In Unit-Tests wird das Interface mit Mockito gemockt — kein LLM, kein Netzwerk, keine Latenz
- **LLM-Integration-Tests:** Das gleiche Interface wird mit `AiServices.builder()` standalone instanziiert — ohne Spring, ohne DB
- **Provider-Austauschbarkeit:** `ChatLanguageModel` ist eine LangChain4j-Abstraktion — ein Wechsel von Ollama zu OpenAI erfordert nur eine Konfigurationsänderung
- **Separation of Concerns:** Business-Logik (Kontext sammeln, speichern, Fehlerbehandlung) ist vollständig von der LLM-Kommunikation getrennt

Constructor Injection durchgängig

Alle Services nutzen **ausschließlich Constructor Injection** — kein `@Autowired` auf Feldern. Das macht Abhängigkeiten explizit und erzwingt, dass alle Abhängigkeiten in Tests bereitgestellt werden:

```
public class PlanningAgentService {
    private final PlanningAiService planningAiService;      // mockbar
    private final EnrichmentAiService enrichmentAiService;  // mockbar
    private final SearxngSearchService searxngSearchService; // mockbar
    private final EmbeddingService embeddingService;        // mockbar
    // ...

    public PlanningAgentService(PlanningAiService planningAiService, ...) {
        this.planningAiService = planningAiService;
        // ...
    }
}
```

DTO-Trennung: Entities niemals in der API

Controller geben nie JPA-Entities zurück — immer **dedizierte Response-Records** mit statischer `from()` - Factory-Methode. Request-DTOs nutzen Bean Validation (`@NotBlank`):

```
// Request – validiert
public record CreateBuddyRequest(
    @NotBlank String name,
    @NotBlank String personality
) {}

// Response – entkoppelt von Entity
public record BuddyResponse(Long id, String name, ...) {
    public static BuddyResponse from(Buddy buddy) { ... }
}
```

Das verhindert, dass interne Datenbankstrukturen in der API sichtbar werden, und erlaubt unabhängige Evolution von API und Schema.

Zentrales Error Handling

Ein `@RestControllerAdvice ( GlobalExceptionHandler )` übersetzt Exceptions in einheitliche JSON-Fehlerantworten:

Exception	HTTP Status	Error Code
<code>EntityNotFoundException</code>	404	<code>NOT_FOUND</code>
<code>InvalidStateException</code>	409	<code>INVALID_STATE</code>
<code>MethodArgumentNotValidException</code>	400	<code>VALIDATION_ERROR</code>
Alle anderen	500	<code>INTERNAL_ERROR</code>

Controller werfen Domain-Exceptions, der Handler kümmert sich um die HTTP-Semantik — keine Try-Catch-Blöcke in Controllern.

Graceful Degradation

Fehlschläge in nicht-kritischen Systemen unterbrechen nie den Hauptprozess:

Fehlerfall	Fallback
Embedding fehlgeschlagen	Task wird trotzdem erstellt/abgeschlossen
Enrichment fehlgeschlagen	Original-Beschreibung wird beibehalten
WordPress nicht erreichbar	Blog-Post nur lokal in DB gespeichert
Websuche fehlgeschlagen	Planung läuft ohne externe Daten
RAG-Abruf fehlgeschlagen	"Keine historischen Daten verfügbar"
Chat-Plan-Analyse fehlgeschlagen	Chat-Antwort wird trotzdem gesendet

Jeder Fallback wird per SLF4J geloggt — kein stiller Fehler, aber auch kein Abbruch.

Testbarkeit auf drei Ebenen

Die Architektur ermöglicht Tests auf jeder Schicht — mit unterschiedlichem Scope und unterschiedlichen Abhängigkeiten:

Controller-Tests (@WebMvcTest)

- MockMvc + gemockte Services
- Testen: HTTP-Status, JSON-Struktur, Validation, Error Handling
- Kein Spring-Context-Start, keine DB

Beispiel: POST /api/buddies mit leerem Name → 400 VALIDATION_ERROR

Service-Tests (@ExtendWith(MockitoExtension.class))

- Alle Abhängigkeiten gemockt (AI, DB, externe Services)
- Testen: Orchestrierung, Fehlerbehandlung, Zustandsübergänge
- Kein Spring, kein Netzwerk

Beispiel: completeTask() bei Embedding-Fehler → Task trotzdem COMPLETED, kein Abbruch

LLM-Integration-Tests (standalone, @Tag("llm"))

- AI Service Interfaces direkt über AiServices.builder()
- Kein Spring, keine DB – nur ChatLanguageModel + Interface
- Testen: LLM-Output-Struktur, Formate, Plausibilität

Beispiel: PlanningAiService → PlannedTasks hat 3-5 Tasks, chronologisch, gültiges Datumsformat

Service-Integration-Tests (standalone, @Tag("integration"))

- Externe Services direkt instanziiert (kein Spring)
- Testen: Erreichbarkeit, Ergebnisqualität

Beispiel: SearxngSearchService → Ergebnisse mit Titel + URL

AI Decision Log

Alle KI-Entscheidungen werden in einer eigenen Tabelle protokolliert — inklusive Kontext (Eingabedaten), Entscheidung (Ergebnis) und Reasoning (Begründung des LLM). Dadurch sind Agenten-Entscheidungen transparent, nachvollziehbar und über die UI einsehbar.

13. Frontend-Architektur

SPA mit React Router

/vbuddy/setup	→ VBuddy erstellen
/vbuddy/buddy/:id/chat	→ Echtzeit-Chat
/vbuddy/buddy/:id/daily-plan	→ Tagesplan-Timeline
/vbuddy/buddy/:id/details	→ Bedürfnisse & Standort
/vbuddy/buddy/:id/ai-log	→ AI Decision Log

Reverse Proxy

Nginx dient als einheitlicher Entry-Point: Statische Dateien unter `/vbuddy/`, API-Requests werden an das Backend proxied — mit SPA-Fallback für Client-Side-Routing.

14. Datenbank-Design & Migrationen

Schema-Evolution via Flyway

6 versionierte Migrationen dokumentieren die Schema-Entwicklung:

Migration	Inhalt
V1__init.sql	Core-Schema: Buddy, Needs, Blog, Chat, AI-Log + pgvector Extension
V2__add_buddy_location.sql	Standort-Tracking
V3__add_vbuddy_task.sql	Task-System mit Status-Management + Indizes
V4__add_buddy_background.sql	Hintergrundgeschichte mit Status-Tracking
V5__add_weekly_schedule.sql	Wochenstundenplan
V6__add_app_user.sql	Benutzerverwaltung mit Rollen (ADMIN/USER)

Bedürfnissystem

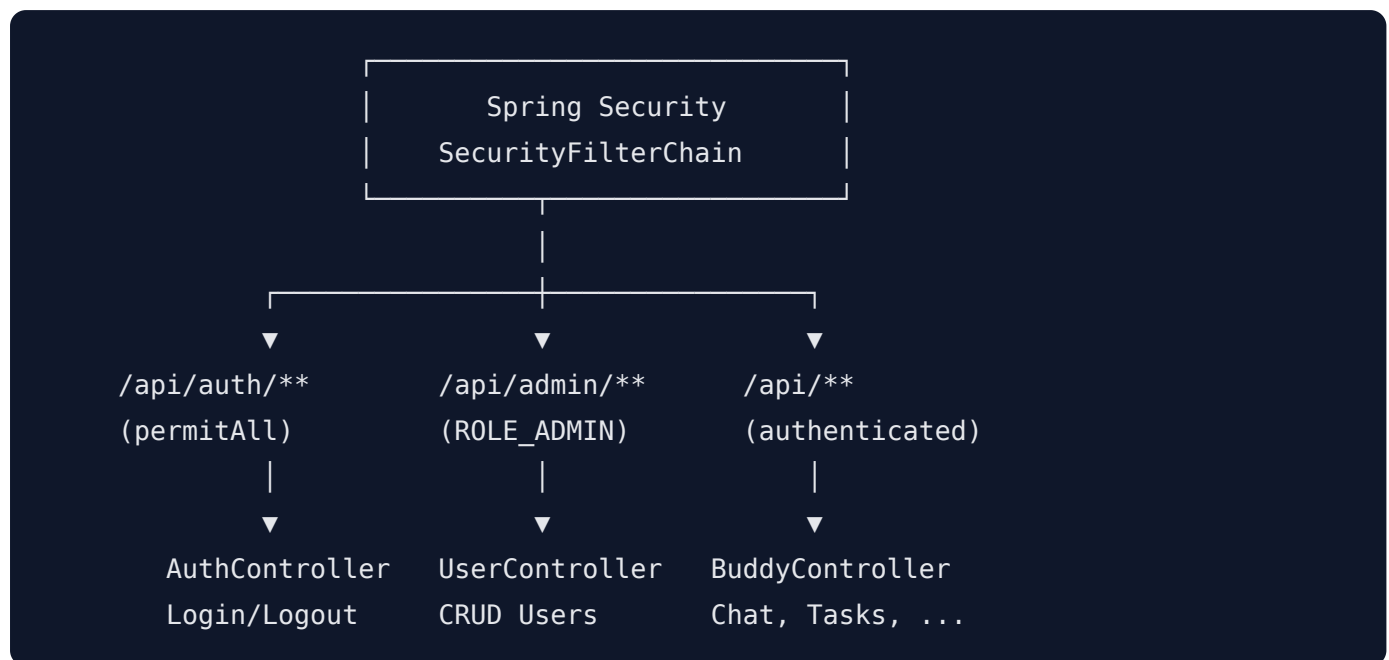
5 Bedürfnistypen (HUNGER, BOREDOM, KNOWLEDGE, EXERCISE, SOCIAL) mit:

- Wertebereich 0–100
- Zeitbasiertem Anstieg (Decay Rate pro Stunde)
- KI-gesteuerter Reduktion durch passende Aktivitäten (-30 bis +10)

15. User Management & Security

Authentifizierung & Autorisierung

VBuddy implementiert eine **session-basierte Authentifizierung** mit Spring Security und rollenbasierter Zugriffskontrolle (RBAC):



Rollenmodell

Rolle	Rechte
ADMIN	Benutzerverwaltung (anlegen, löschen), Vollzugriff auf alle Buddies
USER	Interaktion mit VBuddies (Chat, Tagesplan, Details, AI-Log)

Sicherheitsarchitektur

Backend (Spring Security):

- `AppUserDetailsService` implementiert Spring's `UserDetailsService` zur Benutzer-Authentifizierung
- Passwörter werden mit **BCrypt** gehasht gespeichert
- Session-basierte Authentifizierung mit `SecurityContextHolder`
- `InitialAdminSetup` erstellt beim Start automatisch einen Admin-Benutzer aus Umgebungsvariablen (`ADMIN_USERNAME`, `ADMIN_PASSWORD`)

Frontend (React):

- `useAuth` -Hook als React Context stellt den Auth-State applikationsweit bereit
- `ProtectedRoute` -Komponente schützt alle authentifizierten Routen
- Automatische Redirect-Logik: 401-Antworten leiten zum Login weiter
- Admin-Seite (`/admin`) ist nur für Benutzer mit Rolle ADMIN sichtbar

API-Endpunkte

Methode	Endpunkt	Zugriff	Beschreibung
POST	/api/auth/login	Public	Login mit Username/Passwort
POST	/api/auth/logout	Public	Session invalidieren
GET	/api/auth/me	Public	Aktuellen Benutzer abfragen
GET	/api/users	ADMIN	Alle Benutzer auflisten
POST	/api/users	ADMIN	Neuen Benutzer anlegen
DELETE	/api/users/{id}	ADMIN	Benutzer löschen

Benutzerverwaltung (Admin-UI)

Die Admin-Seite bietet eine Oberfläche zur Benutzerverwaltung:

- Formular zum Anlegen neuer Benutzer (Username, Passwort, Rollenwahl)
- Benutzerliste mit Rollen-Anzeige und Löschen-Funktion
- Fehlerbehandlung bei doppelten Benutzernamen (409 Conflict)

Es gibt **keine Selbstregistrierung** — neue Benutzer werden ausschließlich durch Administratoren angelegt.

Zusammenfassung der Disziplinen

Disziplin	Umsetzung in VBuddy
Multi-Agent-Orchestrierung	Mehrstufige Agenten-Pipelines (4-stufige Background-Pipeline, 3-stufige Task-Pipeline)
Autonome Planung	Lifecycle-Loop mit automatischer Planung, Ausführung, Blog-Veröffentlichung und Bedürfnis-Anpassung
Interaktive Planung	Chat-basierte Planänderungen (CANCEL, UPDATE, ADD) mit Analyse-Agent und False-Positive-Schutz
Retrieval Augmented Generation	pgvector-Embeddings auf 2 Ebenen (Background + Tasks), Abruf in Chat (Top 5) und Planung (Top 10)
Modellstrategie	Aufgabenspezifische LLM-Selektion (Reasoning vs. Creative vs. Conversational), Structured Output, Prompt Engineering
Tool-Augmented Generation	SearxNG-Websuche als Kontextquelle für Agenten
Systemintegration	WordPress REST API für automatische Blog-Veröffentlichung mit Bildern
Teststrategie	Dreistufig: Unit → LLM-Integration → Service-Integration, testbare KI-Architektur
Technologie-Stack	Java 21, Spring Boot 3.4, React 19, TypeScript 5.7, Vite 6, LangChain4j
Cloud-Native Deployment	Docker Compose, Multi-Stage Builds, Netzwerk-Isolation, Healthchecks
On-Premise LLM-Betrieb	Ollama mit 4 verschiedenen LLMs, pgvector für RAG, SearxNG für Suche
Softwarearchitektur	Entkoppelte KI-Schicht (Interface-basiert), Constructor Injection, DTO-Trennung, zentrales Error Handling, Graceful Degradation, testbar auf 4 Ebenen
Datenbank	PostgreSQL 17, Flyway-Migrationen, pgvector-Extension, normalisiertes Schema
Authentifizierung & Autorisierung	Session-basierte Authentifizierung (Spring Security), BCrypt-Passwort-Hashing, RBAC (ADMIN/USER), Admin-UI zur Benutzerverwaltung
Frontend	SPA mit React Router, Nginx Reverse Proxy, Polling für Echtzeit-Updates